Fourier Series

The Fourier Series is a way to represent a periodic function (signal) as a sum of simple sine and cosine waves. It is useful for compression, filtering, edge detection, noise removal, and analysis.

Mathematical Form

$$f(t) = a_0 + \sum_{n=1}^{\infty} \left[a_0 cos\left(\frac{2\pi nt}{T}\right) + b_n sin\left(\frac{2\pi nt}{T}\right) \right]$$

DC term (Average value):

$$a_0 = \int_0^T f(t)\, dt$$

Fourier Series = expresses a periodic function as a sum of sines and cosines.
Fourier Series in image processing = represent an image as a sum of sinusoidal patterns.

Why Fourier in Images?

- An image is just a 2D signal (intensity values across rows & columns).
- Just like a 1D periodic signal (sound wave) can be expressed using Fourier Series, an image (2D signal) can be expressed as a sum of 2D sinusoidal waves.

Applications

- Signal Processing (representing periodic signals)
- Music & Audio compression (Fourier for sound waves)
- Image compression (JPEG uses related Fourier/DCT ideas)
  - ✓ Uses Discrete Cosine Transform (DCT), a Fourier-related transform.
  - ✓ Keeps low-frequency terms (important structure), discards high-frequency (less noticeable).
- Image Filtering
  - ✓ Blurring: keep only low-frequency terms.
  - ✓ Sharpening: enhance high-frequency terms.
- Image Reconstruction
  - ✓ We can approximate an image using only a few Fourier coefficients (like a square wave example).
- Machine learning (Fourier features for kernels)
- Electrical engineering (AC circuits, harmonics)

2D Fourier Series Representation

For a grayscale image $f(x, y)$ with period M in x-direction and N in y-direction:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

This is basically the 2D Fourier Transform, which is the generalized Fourier Series for discrete signals (like digital images).

Interpretation

- Low frequencies (small u, v) → capture smooth variations (overall brightness, background).
- High frequencies (large u, v) → capture edges, textures, fine details.
- Fourier Series breaks an image into smooth part + details.

Example: Let's imagine a 4×4-pixel grayscale image,

$$f(x,y) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 255 & 255 & 0 \\ 0 & 255 & 255 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

This looks like a white square inside black background.

- The Fourier Series expansion would represent this blocky square as a sum of 2D sinusoidal waves (cosines & sines in X and Y directions).
- Most coefficients will be small, except those representing the sharp vertical/horizontal edges (high-frequency terms).

Visualization Idea

- Start with only low-frequency terms → you'll see a blurred version of the square.
- Add more high-frequency terms → edges become sharper → final image looks like the original.

This is exactly like reconstructing a square wave in 1D Fourier Series, but now extended to 2D images.

Summary:

- DFT: transforms image → frequency domain.
- FFT: efficient way to compute DFT.
- IDFT: reconstructs original image.
- In practice: Used for filtering, compression, edge detection, denoising.

What is FFT?

It is an **efficient algorithm** to compute the **Discrete Fourier Transform (DFT)** of a signal or image.

FFT→Fast way to compute DFT.

Purpose of FFT

FFT converts data from the **time (or spatial) domain** into the **frequency domain**. E.g.

- A 1D audio signal → frequency components (pitch, tone)
- A 2D image → frequency components (edges, texture, smooth regions)

Why FFT is Faster

- DFT takes O(N^2) operations.
- FFT reduces this to $O(N\ log\ N)$ using a **divide-and-conquer** approach.

It splits the signal into smaller parts, computes partial transforms, and then combines them.

Applications:

- ✓ Frequency filtering (low-pass, high-pass)
- ✓ Image compression
- ✓ Edge detection
- ✓ Pattern recognition

Image Transforms using DFT

1. Discrete Fourier Transform (DFT):

Operate on **image's Fourier transform** (frequency space). It is converted to image into sinusoidal frequency components. It is useful for **filtering** (low-pass → smoothing, high-pass → edges).

If 2-D linear forward transformations of input image, $f(x,y)$ is $F(u,v)$ and forward transformation kernel is $r(x,y,u,v)$, then we say

$$2-D: F(u,v) = \sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)r(x,y,r,v)$$

Where, $u = 0,1,2,\dots,M-1; and\ v = 0,1,2,\dots,N-1.$
Inverse Transformation,

$$f(x,y) = T(u,v)^{-1}$$

$$f(x,y) = \frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} F(u,v)s(x,y,r,v)$$

***Transform pair*: $T(u,v) = f(x,y)$**

Inverse transformation kernel: $s(x,y,r,v)$
Where, $x = 0,1,2,\dots,M-1; and\ y = 0,1,2,\dots,N-1.$

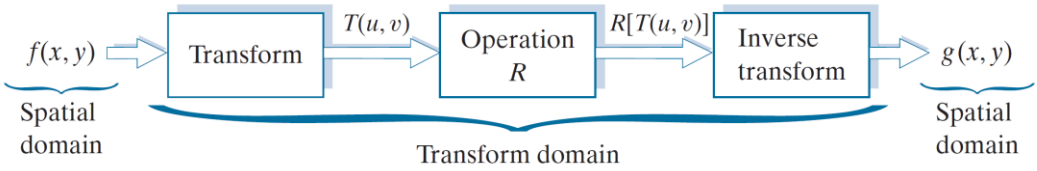
Forward transformation kernel is said to be *separable* if,

$$r(x,y,r,v) = r_1(x,u)r_2(y,v) \Rightarrow r_1(x,u)r_1(y,v)$$

This kernel is said to be symmetric.

Fourier transform forward kernels: $r(x,y,r,v) = e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$

Fourier transform inverse kernels: $s(x,y,r,v) = \frac{1}{MN}e^{j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$



Thus, Discrete Fourier transform (DFT) pair:

$$1D\ DFT: F(u) = \sum_{x=0}^{M-1} f(x)e^{-j2\pi\left(\frac{ux}{M}\right)}$$

$$1D\ IDFT: f(x) = \frac{1}{M}\sum_{u=0}^{M-1} F(u)e^{j2\pi\left(\frac{ux}{M}\right)}$$

$$2D\ DFT: F(u,v) = \sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

$$2D\ IDFT: f(x,y) = \frac{1}{MN}\sum_{u=0}^{M-1}\sum_{v=0}^{N-1} F(u,v)e^{j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

DFT and IDFT pair

$$F(u,v) = F(u+kM) \leftrightarrow f(x) = F(x+kM)$$

$$Convolution\ summation\ (1D): f(x) * h(x) = \sum_{m=0}^{M-1} f(m)h(x-m)$$

$$Convolution\ summation\ (2D): f(x,y) * h(x,y) = \sum_{m=0}^{M-1} f(m,n)h(x-m,y-n)$$

Fourier Spectrum

After applying the **2D Discrete Fourier Transform (DFT)** to an image $f(x,y)$, we get a complex result:

$$F(u,v) = R(u,v) + jI(u,v)$$

Where, $R(u,v)$ is the real part, and $I(u,v)$ is the imaginary part. The **Fourier Spectrum (Magnitude Spectrum)** is the **amplitude** (strength) of each frequency component:

$$|F(u,v)| = \sqrt{R(u,v)^2 + I(u,v)^2}$$

$$For\ very\ large\ scale, Magnitude\ Spectrum = log\ (1+|F(u,v)|)$$

$$Phase\ Spectrum, \theta(u,v) = tan^{-1}\big(F(u,v)\big) = tan^{-1}\left(\frac{I(u,v)}{R(u,v)}\right)$$

Why It's Important in Images?

- Magnitude: Shows *how strong* each frequency is.
- Phase: Contains *position and structure* information.

2. Discrete Cosine Transform (DCT)

- Used in JPEG compression.
- Represents image as cosine basis functions.

3. Wavelet Transform

- Multiresolution analysis.
- Used in JPEG2000.

Applications

- Image compression (JPEG – DCT, JPEG2000 – Wavelet)
- Medical imaging (Fourier filtering in MRI, CT)
- Object recognition (rotation/scale invariance)
- Video stabilization (geometric transformations)
- Watermarking & encryption

Nyquist – Shannon Sampling Theorem for Image Sampling

Sampling rate must be at least twice as high as the highest frequency you want to represent. The sampling theorem guarantees that an analogue signal can be perfectly recovered as long as the sampling rate is at least twice as large as the highest-frequency component of the analogue signal to be sampled. If sampling is too coarse (low sampling rate), high-frequency details of the image appear as false patterns or distortions (aliasing). To avoid aliasing, the **2-D Nyquist Criterion**:

$$F_s >= 2F_{max}$$

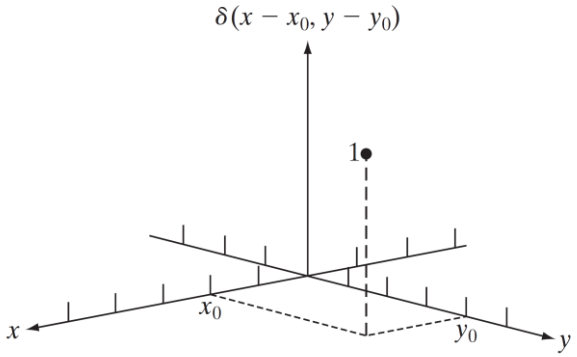
If the highest frequency contained in an analog signal $x_a(t)$ is F_{max}. And the signal is sampled at a rate $F_s >= 2F_{max}$, then $x_a(t)$ can be exactly recovered from its sample values. The sampling rate, $F_S >= 2F_{max}$ is called the Nyquist rate.

The 2-D sampling function is an impulse train:

$$s(x,y) = \sum_m \sum_n \delta(x - mx_m, y - ny_n)$$

and the sampled image is:

$$f_s(x,y) = f(x,y).s(x,y)$$

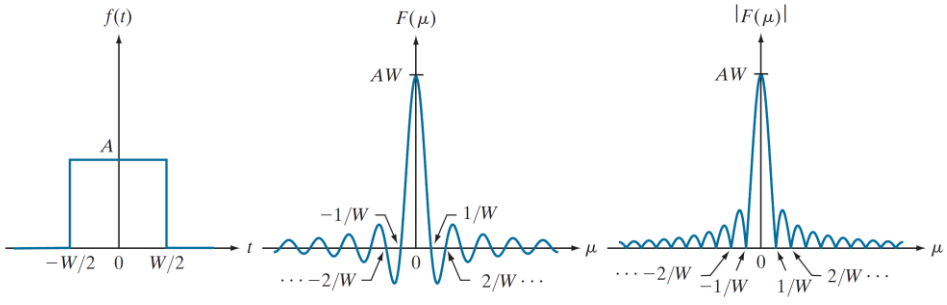


Practical Example

Sampling Rate	Effect on Image
High sampling	Smooth, detailed image
Low sampling	Blocky, distorted (aliasing) image

In Image Processing, Digital cameras perform 2-D spatial sampling with image sensors (CCD/CMOS). Each sensor cell is equal to one sample (pixel).

4.2a##For given a signal. Calculate the Fourier Spectrum.



Solution:

$$F(\mu) = \int_{-\infty}^{\infty} f(t)e^{-j2\pi\mu t}dt = \int_{-W/2}^{W/2} Ae^{-j2\pi\mu t}dt$$

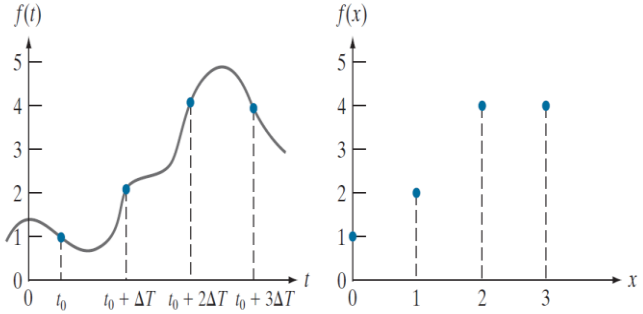
$$= \frac{-A}{j2\pi\mu}\left[e^{-j2\pi\mu t}\right]_{-W/2}^{W/2} = \frac{-A}{j2\pi\mu}\left[e^{-j\pi\mu W} - e^{j\pi\mu W}\right]$$

$$= \frac{A}{j2\pi\mu}\left[e^{j\pi\mu W} - e^{-j\pi\mu W}\right]$$

$$= AW\frac{\sin(\pi\mu W)}{(\pi\mu W)}$$

$\boldsymbol{sinc(m) = \frac{sin(\pi m)}{(\pi m)}}$**, then** $\boldsymbol{|F(u,v)| = AW\left|\frac{sin(\pi\mu W)}{(\pi\mu W)}\right|}$

4.2c## Sampling using DFT and IDFT



$$F(u) = \sum_{x=0}^{M-1} f(x)e^{-j2\pi\left(\frac{ux}{M}\right)}$$

$$\Rightarrow \boldsymbol{F(0)} = \sum_{x=0}^{3} f(x)e^{-j2\pi\left(\frac{0.x}{4}\right)} = \sum_{x=0}^{3} f(x) = f(0) + f(1) + f(2) + f(3) = 1 + 2 + 4 + 4$$
$$= 11$$

F(u)\x	**0**	**1**	**2**	**3**
$\boldsymbol{F(0)}$	$e^{-j0} = 1$	$e^{-j0} = 1$	$e^{-j0} = 1$	$e^{-j0} = 1$
$\boldsymbol{F(1)}$	$e^{-j0} = 1$	$e^{-j\pi/2} = -\mathrm{j}$	$e^{-j\pi} = -1$	$e^{-j3\pi/2} = \mathrm{j}$
$\boldsymbol{F(2)}$	$e^{-j0} = 1$	$e^{-j\pi} = -1$	$e^{-j2\pi} = -1$	$e^{-j3\pi} = -1$
$\boldsymbol{F(3)}$	$e^{-j0} = 1$	$e^{-j3\pi/2} = \mathrm{j}$	$e^{-j3\pi} = -1$	$e^{-j9\pi/2} = -\mathrm{j}$

$$F(1) = \sum_{x=0}^{3} f(x)e^{-j2\pi\left(\frac{x}{4}\right)} = 1 \times e^{0} + 2 \times e^{-j\pi/2} + 4 \times e^{-j\pi} + 4 \times e^{-j3\pi/2} =$$
$$= 1 \times 1 + 2 \times (-j) + 4 \times (-1) + 4 \times j = -3 + 2j$$

$$F(2) = \sum_{x=0}^{3} f(x)e^{-j2\pi\left(\frac{2x}{4}\right)} = f(0) \times e^{-j0} + f(1) \times e^{-j\pi} + f(2) \times e^{-j2\pi} + f(3) \times e^{-j3\pi}$$
$$= 1 \times 1 + 2 \times (-1) + 4 \times 1 + 4 \times (-1) = -1$$

$$F(3) = \sum_{x=0}^{3} f(x)e^{-j2\pi\left(\frac{3x}{4}\right)} = f(0) \times e^{-j0} + f(1) \times e^{-j3\pi/2} + f(2) \times e^{-j3\pi} + f(3) \times e^{-j9\pi/2}$$
$$= 1 \times 1 + 2 \times j + 4 \times (-1) + 4 \times (-j) = -3 - 2j$$

$$\therefore f(0) = \frac{1}{4}\sum_{u=0}^{3} F(u)e^{j2\pi\left(\frac{0}{4}\right)} = \frac{1}{4}\sum_{u=0}^{3} F(u) = \frac{1}{4}[11 - 3 + 2j - 1 - 3 - 2j] = 1$$

4.2d##A simple Image Transform using a small grayscale image and the Discrete Fourier Transform (DFT).

Consider a tiny 2×2 grayscale image:

$$f = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Objective: Compute the 2D Discrete Fourier Transform (DFT) of this image.

Step 1: Recall 2D DFT formula

For an $M \times N$ image $f(x, y)$, the DFT $F[u, v]$ is:

$$F(u, v) = \sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x, y)e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

Where, $u, v = 0,1,\ldots,N-1$ and $j = -1$. For our case, $N = 2$, so $u, v, x, y \in \{0,1\}$.

F(u,v)\(x,y)	**(0,0)**	**(0,1)**	**(1,0)**	**(1,1)**
$\boldsymbol{F(0,0)}$	$e^{-j0} = 1$	$e^{-j0} = 1$	$e^{-j0} = 1$	$e^{-j0} = 1$
$\boldsymbol{F(0,1)}$	$e^{-j0} = 1$	$e^{-j\pi} = -1$	$e^{-j0} = 1$	$e^{-j\pi} = -1$
$\boldsymbol{F(1,0)}$	$e^{-j0} = 1$	$e^{-j0} = 1$	$e^{-j\pi} = -1$	$e^{-j\pi} = -1$
$\boldsymbol{F(1,1)}$	$e^{-j0} = 1$	$e^{-j\pi} = -1$	$e^{-j\pi} = -1$	$e^{-j2\pi} = 1$

Step 2: Compute each $\boldsymbol{F[u,v]}$

$\boldsymbol{F[0,0]}$:

$$F[0,0] = \sum_{x=0}^{1}\sum_{y=0}^{1} f(x,y)e^{-j2\pi(0+0)} = 10 + 20 + 30 + 40 = 100$$

$\boldsymbol{F[0,1]}$:

$$F[0,1] = \sum_{x=0}^{1}\sum_{y=0}^{1} f(x,y)e^{-j2\pi\left(0+\frac{y}{2}\right)} = \sum_{x=0}^{1}\sum_{y=0}^{1} f(x,y)e^{-j2\pi\left(\frac{y}{2}\right)}$$
$$= \left[f[0,0] \times e^{-j2\pi.0} + f[0,1] \times e^{-j2\pi\left(\frac{1}{2}\right)} + f[1,0] \times e^{-j2\pi.0} + f[1,1] \times e^{-j2\pi\left(\frac{1}{2}\right)}\right]$$
$$= [f[0,0] \times 1 + f[0,1] \times (-1) + f[1,0] \times 1 + f[1,1] \times (-1)]$$
$$= [10 \times 1 + 20 \times (-1) + 30 \times 1 + 40 \times (-1)] = -20$$

$\boldsymbol{F[1,0]}$:

Similarly,

$$F[1,0] = \sum_{x=0}^{1}\sum_{y=0}^{1} f(x,y)e^{-j2\pi\left(\frac{x}{2}+0\right)} = \sum_{x=0}^{1}\sum_{y=0}^{1} f(x,y)e^{-j2\pi\left(\frac{x}{2}\right)}$$
$$= \left[f[0,0] \times e^{-j2\pi.0} + f[0,1] \times e^{-j2\pi.0} + f[1,0] \times e^{-j2\pi\left(\frac{1}{2}\right)} + f[1,1] \times e^{-j2\pi\left(\frac{1}{2}\right)}\right]$$
$$= [f[0,0] \times 1 + f[0,1] \times 1 + f[1,0] \times (-1) + f[1,1] \times (-1)]$$
$$= [10 \times 1 + 20 \times 1 + 30 \times (-1) + 40 \times (-1)] = -20$$

$\boldsymbol{F[1,1]}$:

Calculate exponentials for each $f(x,y)$. Thus:

$$F[1,1] = \sum_{x=0}^{1}\sum_{y=0}^{1} f(x,y)e^{-j2\pi\left(\frac{x}{2}+\frac{y}{2}\right)} = [f[0,0] \times e^{0} + f[0,1] \times e^{-j\pi} + f[1,0] \times e^{-j\pi} + f[1,1] \times e^{-j2\pi}]$$
$$= [f[0,0] \times 1 + f[0,1] \times (-1) + f[1,0] \times (-1) + f[1,1] \times 1]$$
$$= [10 \times 1 + 20 \times (-1) + 30 \times (-1) + 40 \times 1] = 0$$

Step 3: Final DFT matrix *F*:

$$F = \begin{bmatrix} 50 & -10 \\ -20 & 0 \end{bmatrix}$$

Interpretation

- $F[0,0]$ is the DC component (average brightness).
- Other components represent different spatial frequencies and phases.
- Negative values are valid complex components (real here because input is real and symmetric).

Summary:

Position	**Value**	**Meaning**
$F[0,0]$	50	Average intensity
$F[0,1]$	-10	Vertical frequency
$F[1,0]$	-20	Horizontal frequency
$F[1,1]$	0	Diagonal frequency

FFT over a Non-separable Kernel

4.3##An FFT over a non-separable kernel example with simple numbers and explain what it means.

What's a "non-separable kernel"?

- A kernel is **separable** if it can be written as the **outer product of two 1D kernels**.
- An**on-separable kernel cannot** be expressed this way.
- FFT (Fast Fourier Transform) is commonly used for efficient **2D convolution** (especially when kernel size is large or non-separable).

Example Setup: Image *f* (3×3).

1	2	3
4	5	6
7	8	9

Non-separable Kernel *K* (3×3)

1	0	-1
1	0	-1
0	0	0

This is similar to a horizontal edge detection filter but **not separable** because it can't be written as a product of two vectors.

Step 1: Zero-pad both to same size (e.g., 5×5)
Why? Because FFT-based convolution assumes **circular convolution**, so padding helps prevent wrap-around artifacts.
We pad both *I* and *K* to **5×5** by adding zeros.

Padded $I_{5\times5}$, I_p:

1	2	3	0	0
4	5	6	0	0
7	8	9	0	0
0	0	0	0	0
0	0	0	0	0

Padded $K_{5\times5}$, K_p:

1	0	-1	0	0
1	0	-1	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Step 2: Compute 2D FFT of both
Let:

$$F_I = FFT2(Ip), F_K = FFT2(Kp)$$

We won't compute full FFTs due to complexity, but let's note:

- FFT2 gives a 5×5 matrix of complex values.
- Element-wise multiplication in frequency domain:

$$F_R = F_I . F_K$$

Step 3: Inverse FFT

Apply:

$$R = IFFT2(F_R)$$

This gives you the **convolution result** of the original image with the non-separable kernel.

Final Output:

- The result *R* will be a **5×5** filtered image.
- You can **crop** it back to the original size (3×3) if needed.

Why Use FFT for Non-Separable Kernels?

- Normal convolution: $O(n^2m^2)$ for image and kernel of size $n \times n, m \times m$
- FFT-based: $O(n^2 \log n)$, **much faster** for large filters—even non-separable ones.

Summary:

Step	Action
1	Pad image and kernel to same size (e.g., 5×5)
2	Compute 2D FFT of both
3	Multiply element-wise in frequency domain
4	Apply inverse FFT to get spatial result
5	Crop if needed to original size

M-point Discrete Window 1-D FFT

4.4##Let's do a simple 4-point Discrete Window 1-D FFT.

Step 1: Define input signal (length M=4)

$$x = [1,\ 2,\ 3,\ 4]$$

Step 2: Define FFT formula

For $k = 0,1,2,3,$

$$X(k) = \sum_{n=0}^{3} x(n) \cdot e^{-j\frac{2\pi}{4}kn} = x(0) \cdot e^{-j\frac{2\pi}{4}k.0} + x(1) \cdot e^{-j\frac{2\pi}{4}k.1} + x(2) \cdot e^{-j\frac{2\pi}{4}k.2} + x(3) \cdot e^{-j\frac{2\pi}{4}k.3}$$

$$= x(0) + x(1) \cdot e^{-j\frac{2\pi}{4}k.1} + x(2) \cdot e^{-j\frac{2\pi}{4}k.2} + x(3) \cdot e^{-j\frac{2\pi}{4}k.3}$$

Step 3: Calculate each *X*(*k*)

$for\ k = 0, n = 0:\ e^0 = 1$

$for\ k = 1, n = 1:$

$$e^{-j\frac{2\pi}{4}1.1} = e^{-\frac{j2\pi}{4}} = e^{-\frac{j\pi}{2}} = \cos\left(\frac{\pi}{2}\right) - \mathrm{j}\sin\left(\frac{\pi}{2}\right) = 0 - j1 = -j$$

$$e^{-\frac{j\pi}{2}} = -j,\ \left(e^{-j\frac{\pi}{2}}\right)^2 = (-j)^2 = -1,\ \left(e^{-j\frac{\pi}{2}}\right)^3 = (-j)^3 = j, \left(e^{-j\frac{\pi}{2}}\right)^4 = (-j)^4 = 1,$$

$$\left(e^{-j\frac{\pi}{2}}\right)^6 = (-j)^6 = -1, \left(e^{-j\frac{\pi}{2}}\right)^9 = (-j)^9 = -j$$

***k*=0:**

$$X(k) = x(0) + x(1) \cdot e^{-j\frac{2\pi}{4}k.1} + x(2) \cdot e^{-j\frac{2\pi}{4}k.2} + x(3) \cdot e^{-j\frac{2\pi}{4}k.3}$$

$$\Rightarrow X(0) = x(0) + x(1) + x(2) + x(3) = 1 + 2 + 3 + 4 = 10$$

***k*=1:**

$$X(k) = x(0) + x(1) \cdot e^{-j\frac{2\pi}{4}k.1} + x(2) \cdot e^{-j\frac{2\pi}{4}k.2} + x(3) \cdot e^{-j\frac{2\pi}{4}k.3}$$

$$\Rightarrow X(1) = x(0) + x(1) \cdot e^{-j\frac{2\pi}{4}1.1} + x(2) \cdot e^{-j\frac{2\pi}{4}1.2} + x(3) \cdot e^{-j\frac{2\pi}{4}1.3}$$

$$= x(0) + x(1) \cdot e^{-j\frac{\pi}{2}} + x(2) \cdot \left(e^{-j\frac{\pi}{2}}\right)^2 + x(3) \cdot \left(e^{-j\frac{\pi}{2}}\right)^3$$

$$= 1 + 2(-j) + 3(-1) + 4(j) = 1 - 2j - 3 + 4j = -2 + 2j$$

***k*=2:**

$$X(k) = x(0) + x(1) \cdot e^{-j\frac{2\pi}{4}k.1} + x(2) \cdot e^{-j\frac{2\pi}{4}k.2} + x(3) \cdot e^{-j\frac{2\pi}{4}k.3}$$

$$\Rightarrow X(2) = x(0) + x(1)\cdot e^{-j\frac{2\pi}{4}2.1} + x(2)\cdot e^{-j\frac{2\pi}{4}2.2} + x(3)\cdot e^{-j\frac{2\pi}{4}2.3}$$
$$= x(0) + x(1)\cdot\left(e^{-j\frac{\pi}{2}}\right)^2 + x(2)\cdot\left(e^{-j\frac{\pi}{2}}\right)^4 + x(3)\cdot\left(e^{-j\frac{\pi}{2}}\right)^6$$
$$= 1 + 2(-1) + 3(1) + 4(-1) = 1 - 2 + 3 - 4 = -2$$

***k*=3:**

$$X(k) = x(0) + x(1)\cdot e^{-j\frac{2\pi}{4}k.1} + x(2)\cdot e^{-j\frac{2\pi}{4}k.2} + x(3)\cdot e^{-j\frac{2\pi}{4}k.3}$$
$$\Rightarrow X(3) = x(0) + x(1)\cdot e^{-j\frac{2\pi}{4}3.1} + x(2)\cdot e^{-j\frac{2\pi}{4}3.2} + x(3)\cdot e^{-j\frac{2\pi}{4}3.3}$$
$$= x(0) + x(1)\cdot\left(e^{-j\frac{\pi}{2}}\right)^3 + x(2)\cdot\left(e^{-j\frac{\pi}{2}}\right)^6 + x(3)\cdot\left(e^{-j\frac{\pi}{2}}\right)^9$$
$$= 1 + 2(j) + 3(-1) + 4(-j) = 1 + 2j - 3 - 4j = -2 - 2j$$

Step 4: Final FFT output

$$X = [10,\ -2 + 2j,\ -2,\ -2 - 2j]$$

2D FFT on 2×2 Grayscale Image

4.5##Applying the 2D Fast Fourier Transform (FFT) on a 2×2 grayscale image along with a visual explanation using simple numbers.

Original Image (2×2):

$$f = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

The **2D DFT** of a 2×2 image is for an $M \times N$ image $f[x, y]$, the DFT $F[u, v]$ is,

$$F(u,v) = \sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

Where:

- $u, v = 0,1,\ldots,N-1$
- $j = -1$

For our case, $N = 2$, so $u, v, x, y \in \{0,1\}$.

$$F(u,v) = \sum_{x=0}^{1}\sum_{y=0}^{1} f(x,y)e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

We'll compute all $F(u,v)\ for\ u,v = 0\ or\ 1$.

Step 1: Compute *F*(0,0)

$$F(0,0) = 1 + 2 + 3 + 4 = 10$$

Step 2: Compute *F*(0,1)

$$F(0,1) = (1)e^{-j0} + (2)e^{-j\pi} + (3)e^{-j0} + (4)e^{-j\pi}$$
$$= (1)(1) + (2)(-1) + (3)(1) + (4)(-1) = -2$$

Step 3: Compute *F*(1,0)

$$F(1,0) = (1)e^{-j0} + (2)e^{-j0} + (3)e^{-j\pi} + (4)e^{-j\pi}$$
$$= (1)(1) + (2)(1) + (3)(-1) + (4)(-1) = -4$$

Step 4: Compute *F*(1,1)

$$F(1,1) = (1)e^{-j0} + (2)e^{-j\pi} + (3)e^{-j\pi} + (4)e^{-j2\pi}$$
$$= (1)(1) + (2)(-1) + (3)(-1) + (4)(1) = 0$$

Final FFT Output Matrix:

$$F = \begin{bmatrix} 10 & -2 \\ -4 & 0 \end{bmatrix}$$

Interpretation:

- **F(0,0)** = 10 is the **DC component** (average brightness).
- Others are **frequency components** showing contrast, edges, and patterns.

Image DFT and IDFT

4.6##A 2D DFT was found for the given input and calculate IDFT on DFT output.

DFT Output:

$$F(u,v) = \begin{bmatrix} 10 & -2 \\ -4 & 0 \end{bmatrix}$$

Step 4: Inverse DFT (IDFT)

$$f(x,y) = \frac{1}{M \times N} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) e^{j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

Here, $M = N = 2$, so divide by 4 and use $e^{j0} = 1$ and $e^{j\pi} = -1$.

F(u,v)\(x, y)	**(0,0)**	**(0,1)**	**(1,0)**	**(1,1)**
$\boldsymbol{F(0,0)}$	$e^{j0} = 1$	$e^{j0} = 1$	$e^{j0} = 1$	$e^{j0} = 1$
$\boldsymbol{F(0,1)}$	$e^{j0} = 1$	$e^{j\pi} = -1$	$e^{j0} = 1$	$e^{j\pi} = -1$
$\boldsymbol{F(1,0)}$	$e^{j0} = 1$	$e^{j0} = 1$	$e^{j\pi} = -1$	$e^{j\pi} = -1$
$\boldsymbol{F(1,1)}$	$e^{j0} = 1$	$e^{j\pi} = -1$	$e^{j\pi} = -1$	$e^{j2\pi} = 1$

Step 5: Reconstruct of *f(x,y)*

$$f(0,0) = \frac{1}{2 \times 2} \sum_{u=0}^{1} \sum_{v=0}^{1} F(u,v) = \frac{10 + (-2) + (-4) + 0}{4} = 1$$

$$f(0,1) = \frac{1}{2 \times 2} \sum_{u=0}^{1} \sum_{v=0}^{1} F(u,v) e^{j2\pi\left(\frac{v}{2}\right)} = \frac{10e^{j0} + (-2)e^{j\pi} + (-4)e^{j0} + 0}{4} = 2$$

$$f(1,0) = \frac{1}{2 \times 2} \sum_{u=0}^{1} \sum_{v=0}^{1} F(u,v) e^{j2\pi\left(\frac{u}{2}\right)} = \frac{10e^{j0} + (-2)e^{j0} + (-4)e^{j\pi} + 0}{4} = 3$$

$$f(1,1) = \frac{1}{2 \times 2} \sum_{u=0}^{1} \sum_{v=0}^{1} F(u,v) e^{j2\pi\left(\frac{u}{2}+\frac{v}{2}\right)} = \frac{10e^{j0} + (-2)e^{j\pi} + (-4)e^{j\pi} + 0}{4} = 4$$

Final Reconstructed Image:

$$f(x,y) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Perfectly matches the original.

Summary:

Step	Result
DFT	Converts image to frequency domain
IDFT	Reconstructs the original image
Accuracy	Perfect match (no loss)
Limitation	Small size shown; large images use FFT algorithms